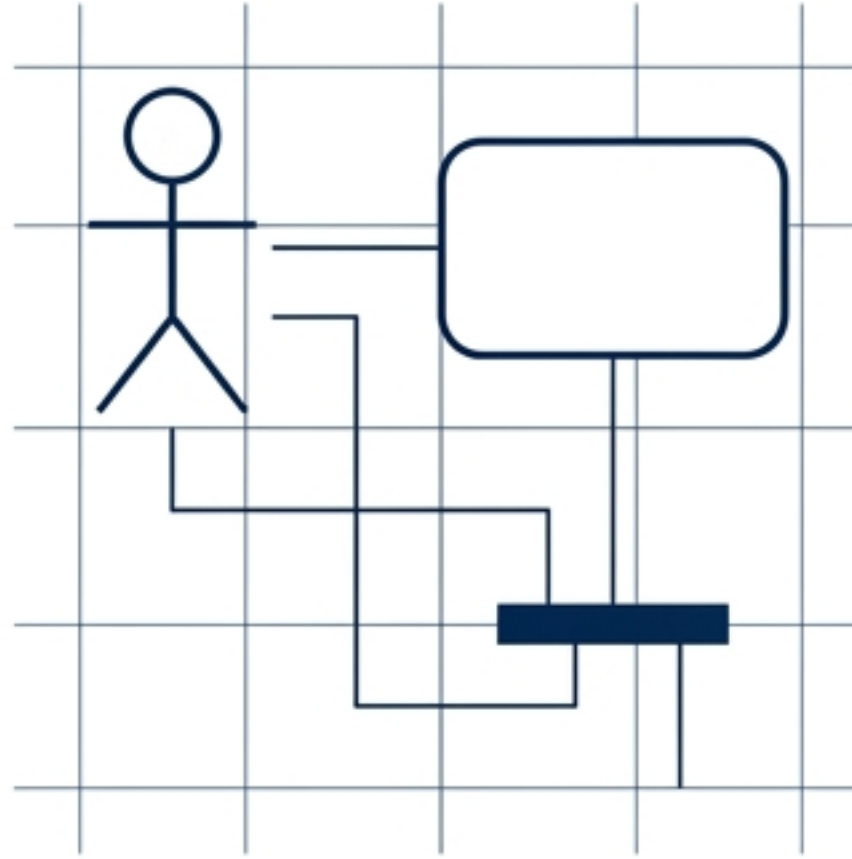




UML Diagram Taxonomy

A Student's Guide to System Modeling
Advanced Software Architecture Lab

Lab Objectives



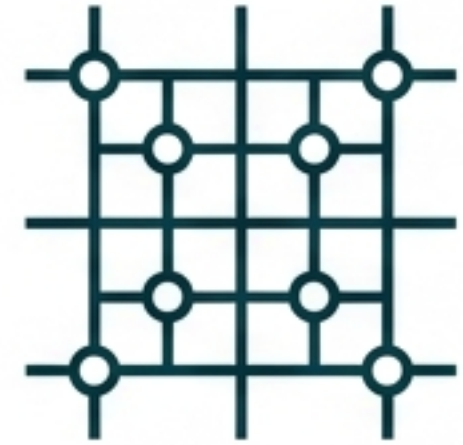
Master Abstraction

Understand how to deliberately simplify systems to highlight salient characteristics.



Apply Core Frameworks

Navigate the 4 essential perspectives and the "Big Five" UML diagrams.



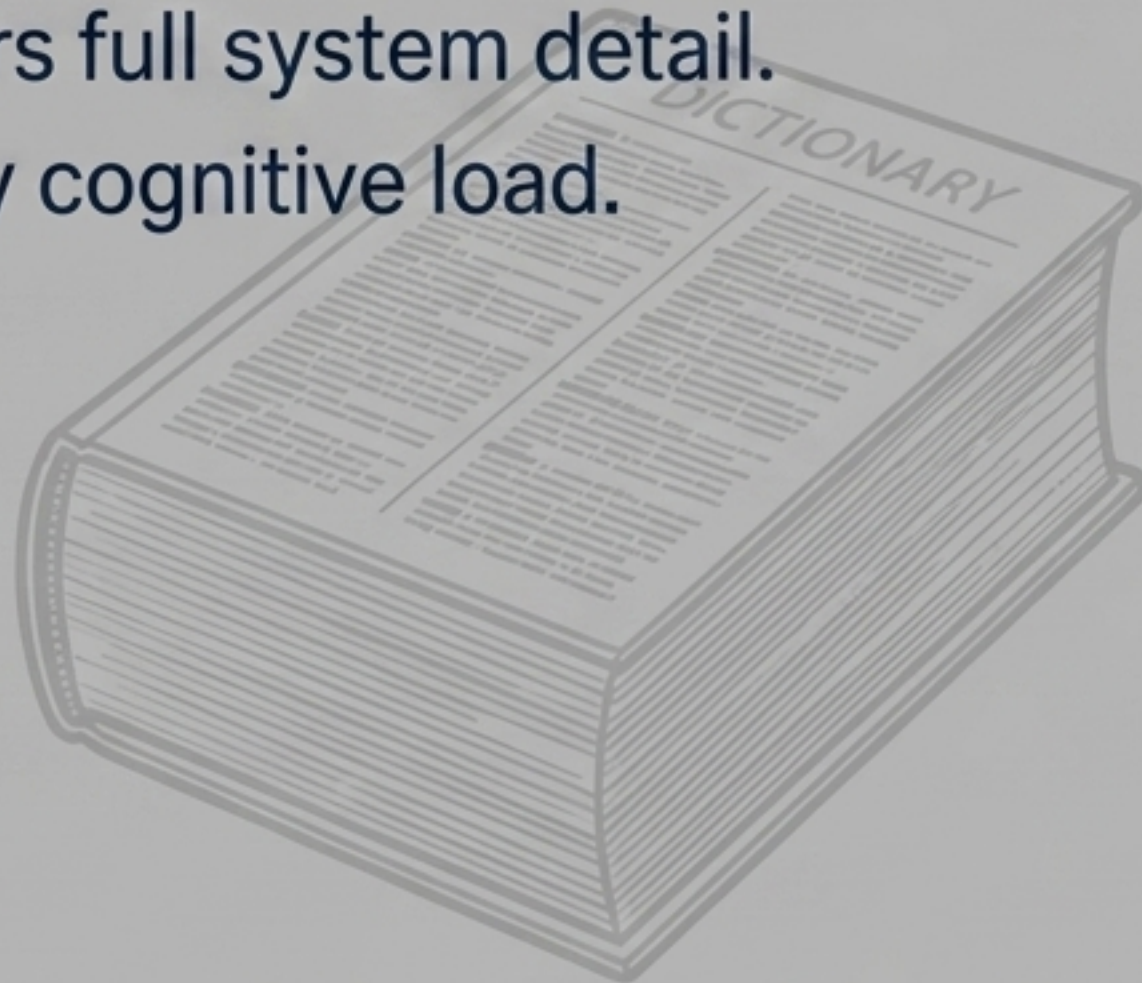
Execute Models

Map contextual boundaries, internal processes, and specific component interactions.

The Master Tool: Abstraction

Representation

- Goal: Complete translation.
- Maintains all original information.
- Mirrors full system detail.
- Heavy cognitive load.

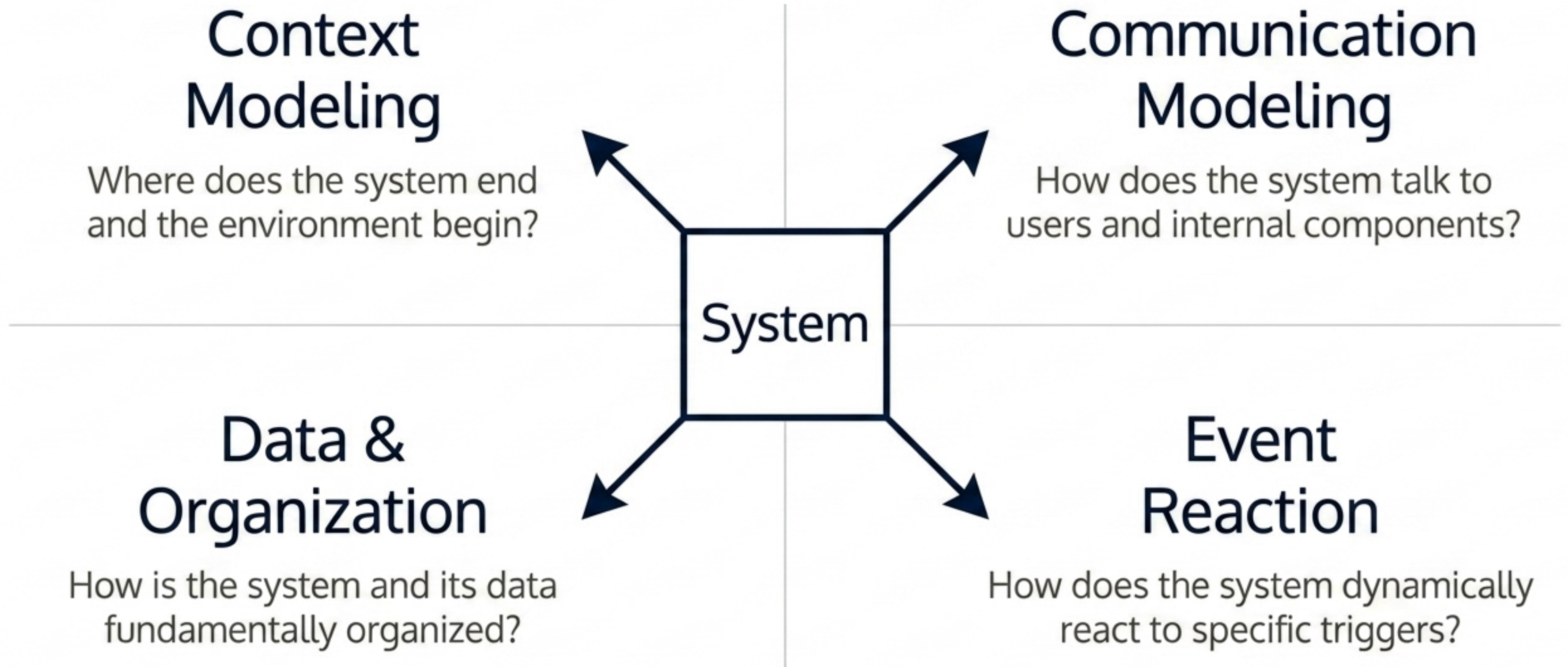


Abstraction

- Goal: Strategic summary.
- Deliberate simplification.
- Highlights only salient traits.
- The foundation of UML modeling.



The Four Essential Lenses

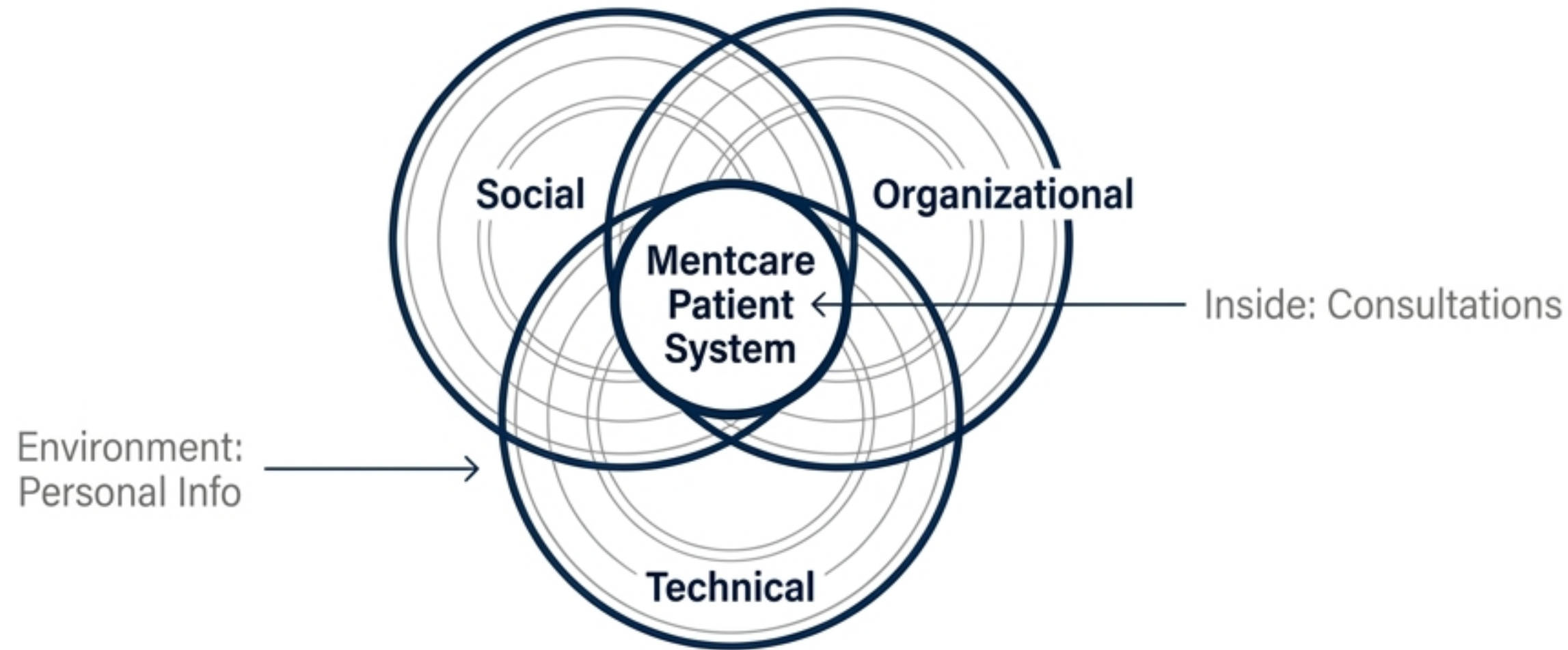


The "Big Five" Industry Standard

Diagram	Primary Focus	Key Benefit
Activity	Process & data flow	Maps internal business workflows
Use Case	System & environment	Defines user requirements & boundaries
Sequence	Actor & component interaction	Visualizes time-ordered communication
Class	Object associations	Identifies structural organization
State	Event reactions	Models dynamic behavioral shifts

Step 1 | Define System Boundaries

Establish what lives inside the software versus the environment.
Boundaries are shaped by three non-obvious forces:



Social Factors:
Diverse user needs (e.g., teachers vs. children).

Organizational Factors:
Management preferences or departmental scope.

Technical Factors:
Duplicating data for speed vs. relying on external systems.

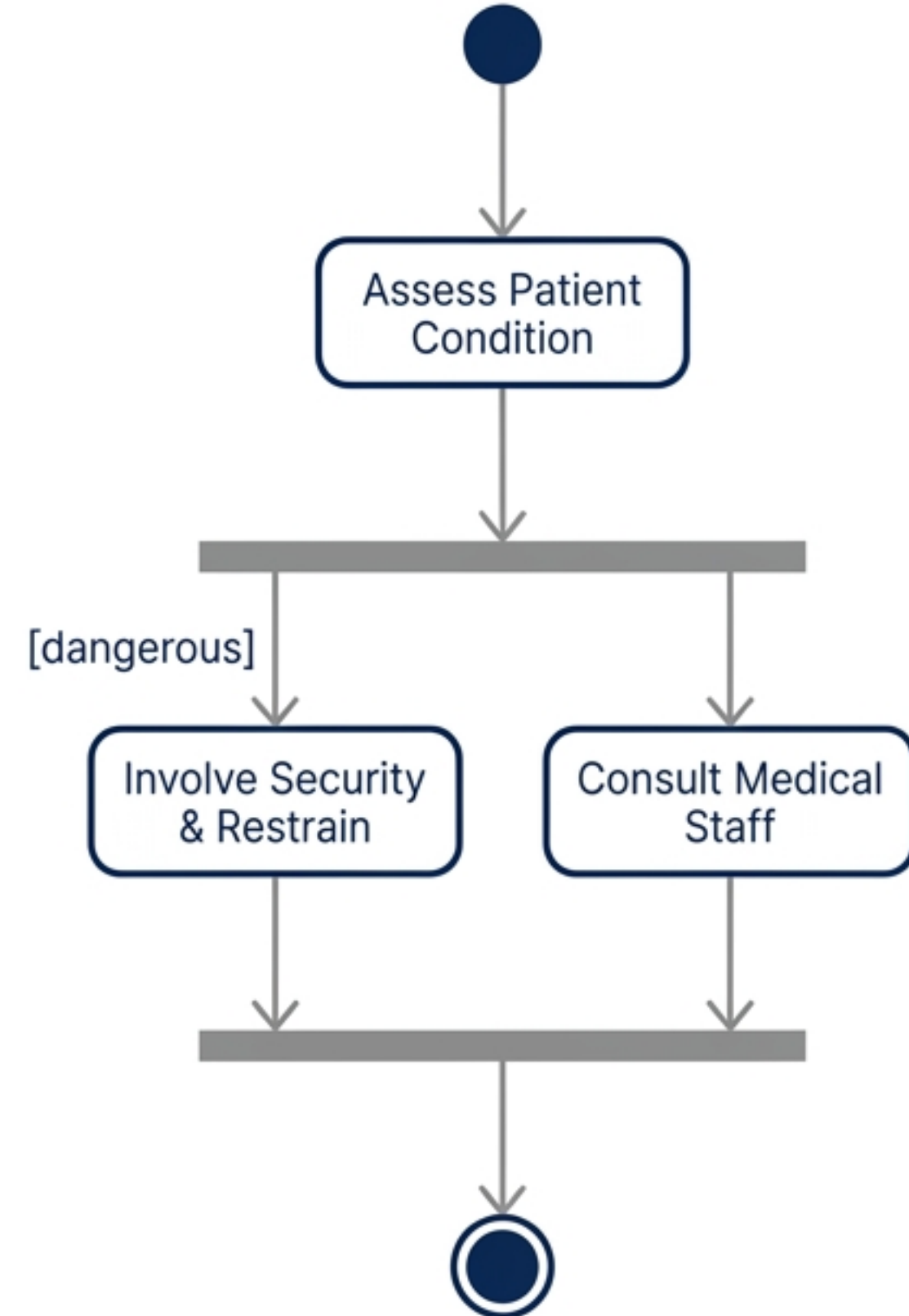
Step 2 | Map Business Processes

Map the high-level flow of control before defining internal logic using an Activity Diagram.

Notation Guide:

-  Filled Circle: Process start
-  Double Circle: Process end
-  Rounded Rectangle: Specific activity
-  Solid Bar: Activity coordination (parallel execution)
- [Guards] Work flow conditions (e.g., [dangerous])

Involuntary Patient Detention Process



Step 3 | Use Case vs. Sequence

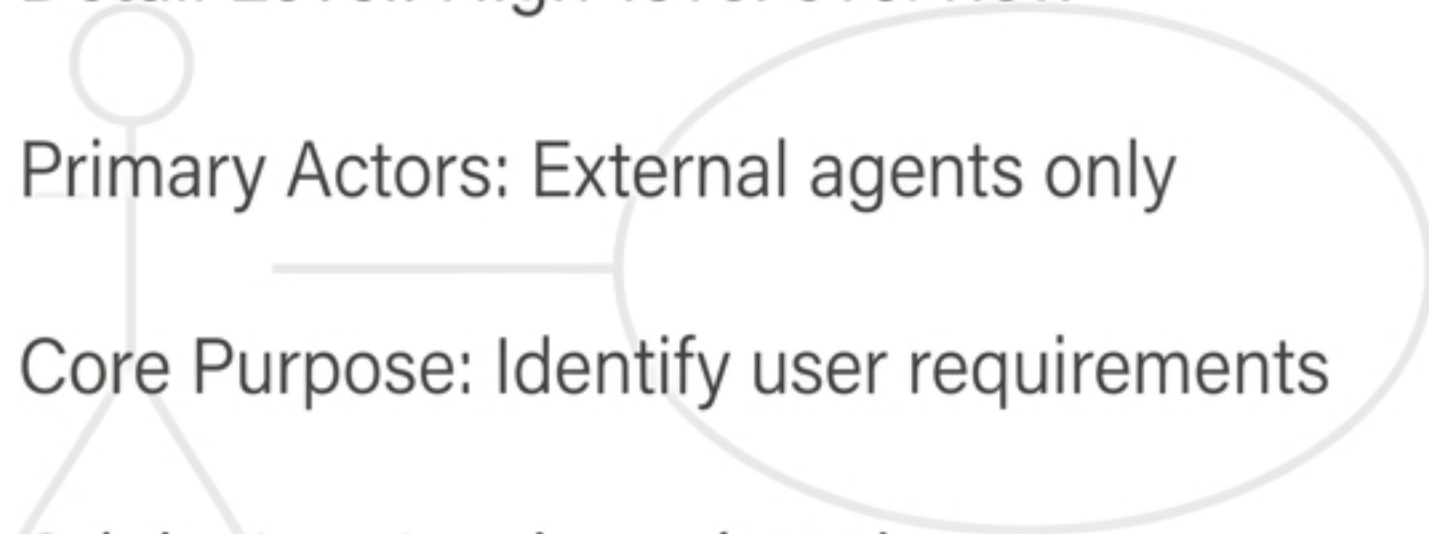
Use Case Diagrams

Detail Level: High-level overview

Primary Actors: External agents only

Core Purpose: Identify user requirements

Origin: Ivar Jacobsen (1993)



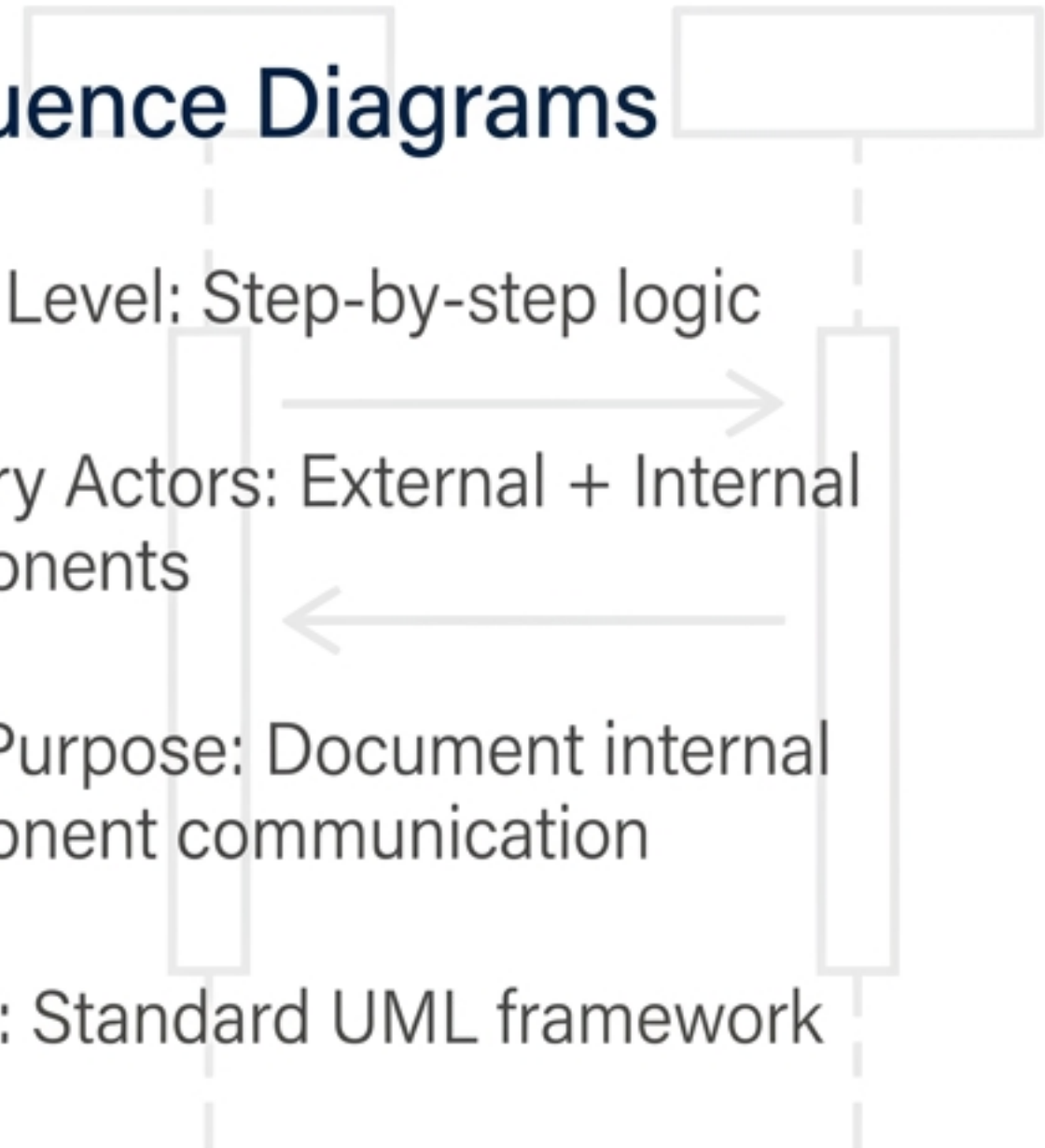
Sequence Diagrams

Detail Level: Step-by-step logic

Primary Actors: External + Internal components

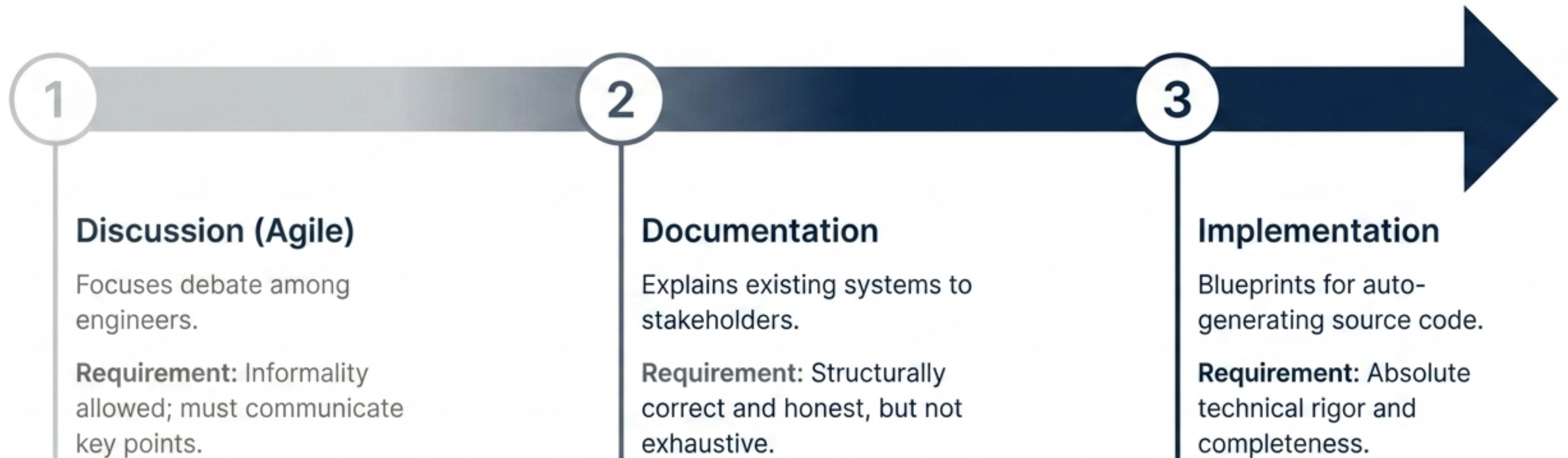
Core Purpose: Document internal component communication

Origin: Standard UML framework



Step 4 | Determine Rigor Level

Determine your diagram's required rigor based on its lifecycle phase:



Lab Deliverables |

For your project, you must submit:



Context Model: Justification of social/technical boundaries.



Activity Diagram: Mapping one core automated process.



Use Case Diagram: High-level user requirements.



Sequence Diagram: Detailed component logic for one use case.

Common Pitfalls



Over-detailing Early Models

Attempting to make initial models exhaustive. Remember: omitting detail is the primary feature of a good abstraction.



Misaligning Form and Function

Using an Implementation-level diagram for an Agile discussion, causing massive, unnecessary cognitive overhead.



Ignoring Non-Technical Boundaries

Defining system context strictly by technology, while ignoring organizational politics or unique social needs.

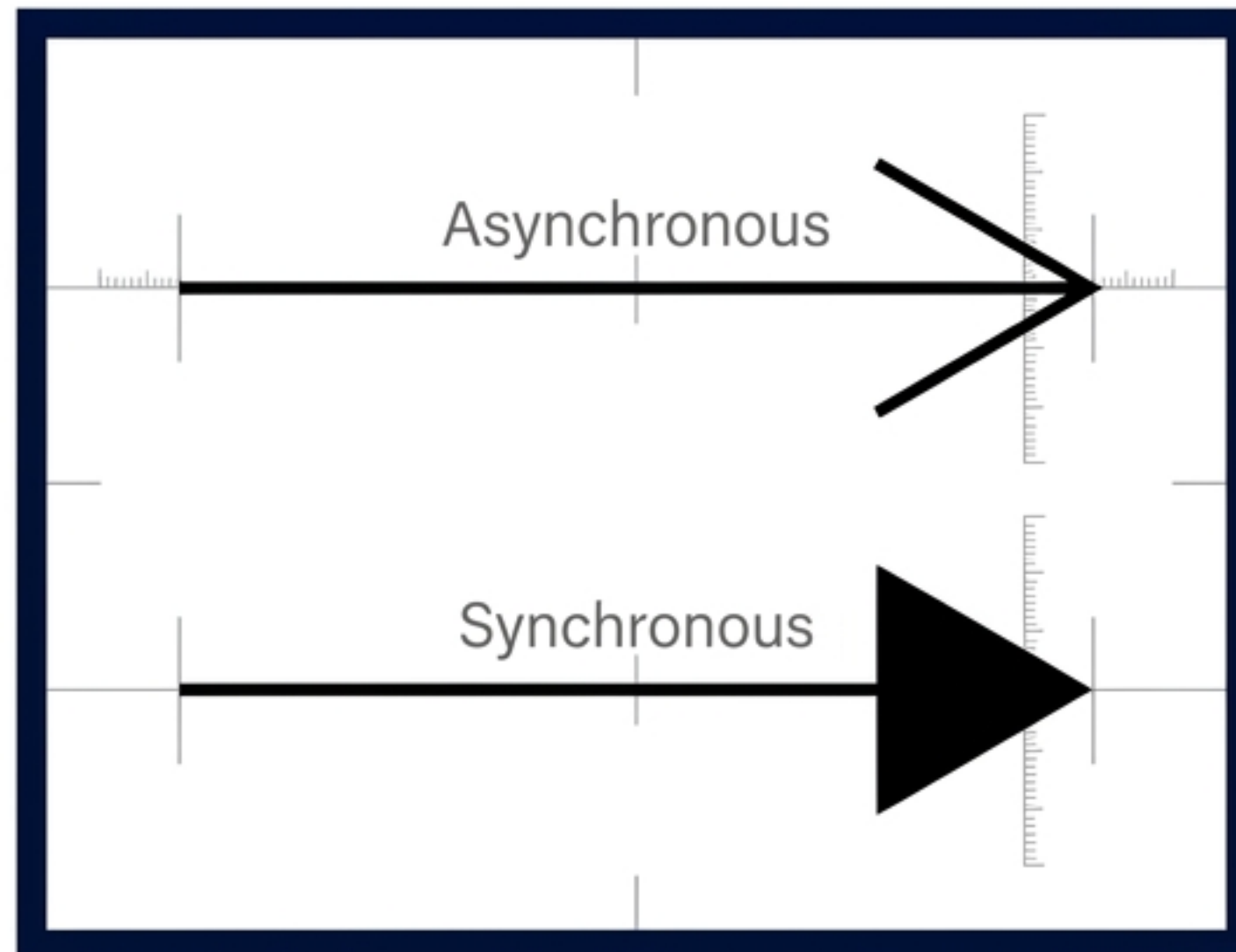
Bonus Challenge: Model-Driven Engineering

The Task: Create an Implementation-level Sequence diagram.

In Model-Driven Engineering (MDE), models automatically generate source code.

The Catch: Absolute precision is required.

Example: You must perfectly distinguish between stick arrowheads (asynchronous) and block arrowheads (synchronous). A visual mistake here fundamentally alters the generated software logic.



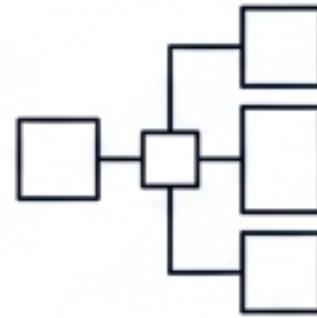
Summary Synthesis

A diagram's form must always follow its function.



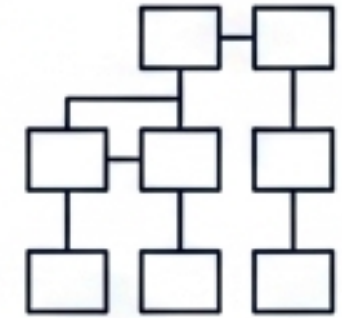
1. Perspective First

Identify if you need an external, interactive, structural, or behavioral lens before drawing.



2. Abstract Wisely

Use the "Big Five" diagrams to simplify complex reality into salient traits.



3. Match the Rigor

Scale your exactness based on whether you are debating, documenting, or deploying code.